P3283. Adding .first() and .last() to strings

Draft Proposal, 15 May 2024

This version:
TBA

Editor:
rhidiandewit@gmail.com

Abstract

C++ has a lot of nice utility functions in its `std::string` library, but the richness of the API can be even more improved. This proposal aims to add `.first()` and `.last()` functions to easily access the first N and last N characters of a string.

Table of Contents

6.2 `std::basic_string::first`

6.3 `std::basic_string::last`

6.4 Addition to `<string_view>`

6.5 `std::basic_string_view::first`

6.6 `std::basic_string_view::last`

§ 1. Table of Contents

§ 2. Changelog

§ 2.1. R3

- Remove `noexcept` from `std::basic_string::first()` and `std::basic_string::last()`
- Remove template parameters from proposed wording

- Added a `const &` and `&&` overload for `std::basic_string::first()` and `std::basic_string::last()`
- Adjust Abstract

§ 2.2. R2

- Add similarities to `std::span` in Motivation and Scope

§ 2.3. R1

- Add more to Motivation and Scope
- Add Technical Specifications
- Add Proposed Wording

§ 2.4. R0

- Initial revision

§ 3. Motivation and Scope

The `std::string` (and `std::string_view`) api has a lot of nice features and utility functions, but can be improved even further. Currently, to get the first `N` characters of a string, we need to do `std::string::substr(0, N)`. This is fine, but people might be confused as to the usage of the `0` in the function call. Similarly, to get the last `N` characters of a string, we need to do `std::string::substr(size() - N, N)`. This is also fine, but again, the first parameter could lead to confusion, or even a possible error if the wrong number were passed.

To facilitate these simple operations, I suggest adding `std::string::first()` and `std::string::last()` (and similar counterparts to `std::string_view`). Having these 2 utility functions brings a couple of benefits:

1. It is more obvious and clear what the programmer intent is, `.first(N)` conveys better meaning as opposed to `.substr(0, N)`, same for `.last(N)` and `.substr(size() - N, N)`
2. Since there are less arguments, and the argument is only a count, there is less potential for programmer error

3. It is generally simpler and less typing.

There is the argument of the `std::string` API already being far too large, and this being only a minor addition. Although this is true, the `std::string` API has some weird missing features, and the richness of the API is important, currently, programmers are often forced to make small utility functions themselves to do trivial operations on a string that should be part of the STL.

There is also already existing similar functionality in `std::span`. `std::span::first` and `std::span::last` already exist and implement the feature proposed in this paper, even though there is also the `std::span::subspan` function, suggesting that `std::span::first()` and `std::span::last()` had enough interest to be added besides `std::span::subspan()`. I believe it should be the same treatment for `std::string` and `std::string_view`, which only have `.substr()`.

§ 4. Impact on the Standard

These are 2 minor utility functions to be added to the `std::string` and `std::string_view` API, not impacting any existing code, so the impact on the standard is minimal.

§ 5. Technical Specifications

1. `std::basic_string::first()` takes 1 parameter: `size_t count` and returns a `std::basic_string`.
    - `count` is the amount of characters to included, starting from the start of the original string.
        - Determines the effective length of `count` as `std::min(count, size())`
2. `std::basic_string::last()` takes 1 parameter: `size_t count` and returns a `std::basic_string`.
    - `count` is the amount of characters to be included, starting from the end of the original string.
        - Determines the effective length of `count` as `std::min(count, size())`
3. `std::basic_string_view::first()` takes 1 parameter: `size_t count`, returns a `std::basic_string_view` and has a `noexcept` guarantee.
    - `count` is the amount of characters to included, starting from the start of the original string_view.

- Determines the effective length of `count` as `std::min(count, size())`

4. `std::basic_string_view::last()` takes 1 parameter: `size_t count`, returns a `std::basic_string_view` and has a `noexcept` guarantee.
    - `count` is the amount of characters to be included, starting from the end of the original string_view.
        - Determines the effective length of `count` as `std::min(count, size())`

§ 6. Proposed Wording

§ 6.1. Addition to `<string>`

Add the following to 23.4.3.1 basic.string.general:

```
// [...]
namespace std {
  // [...]

  // [string.ops], string operations
  // [...]

  constexpr basic_string first(size_t count) const &;
  constexpr basic_string first(size_t count) &&;

  constexpr basic_string last(size_t count) const &;
  constexpr basic_string last(size_t count) &&;
}
```

§ 6.2. `std::basic_string::first`

Add the following subclause to 23.4.3.8 string.ops:

- 23.4.3.?: `basic_string::first [string.first]`
    - `constexpr basic_string first(size_t count) const;`
        1. *Effects*: Determines the effective length `xlen` of the string to be returned as `std::min(count, size())`.

Returns the characters in the range `[data(), data() + xlen);`

2. *Returns*: `basic_string(data(), data() + xlen)`

§ 6.3. `std::basic_string::last`

Add the following subclause to 23.4.3.8 string.ops:

- 23.4.3.?: `basic_string::last` `[string.last]`
  - `constexpr basic_string last(size_t count) const;`
    1. *Effects*: Determines the effective length `xlen` of the string to be returned as `std::min(count, size())`.
       Returns the characters in the range `[data() + size() - xlen, xlen);`
    2. *Returns*: `basic_string(data() + size() - xlen, data() + size())`

§ 6.4. Addition to `<string_view>`

Add the following to 23.3.3.1 string.view.template.general:

```
// [...]
namespace std {
  // [...]

  // [string.ops], string operations
  // [...]

  constexpr basic_string_view first(size_t count) const noexcept;
  constexpr basic_string_view last(size_t count) const noexcept;
}
```

§ 6.5. `std::basic_string_view::first`

Add the following subclause to 23.3.3.8 string.view.ops:

- 23.3.3.?: `basic_string_view::first` `[string.first]`
  - `constexpr basic_string_view first(size_t count) const noexcept;`

1. *Effects*: Determines the effective length `xlen` of the string to be returned as `std::min(count, size())`.
   Returns the characters in the range `[data(), data() + xlen);`
2. *Returns*: `basic_string_view(data(), data() + xlen)`

§ 6.6. `std::basic_string_view::last`

Add the following subclause to 23.3.3.8 string.view.ops:

- 23.3.3.?: `basic_string_view::last` `[string.last]`
  - `constexpr basic_string_view last(size_t count) const noexcept;`
    1. *Effects*: Determines the effective length `xlen` of the string to be returned as `std::min(count, size())`.
       Returns the characters in the range `[data() + size() - xlen, xlen);`
    2. *Returns*: `basic_string_view(data() + size() - xlen, data() + size())`